

Aura Lite

Post-Training a Customer Support
Decision Specialist with GRPO

GRPO reinforcement learning on Qwen2.5-1.5B
7-component reward function | 266 training scenarios | LoRA adapters

Ahmad Naggayev | September 2026

The Problem

A customer messages support about a broken phone. The AI agent must decide what to do next -- not generate text, but make a **triage decision**.

6 possible actions:

Action	Real-World Meaning
ASK_CLARIFICATION	"Can you tell me more?" -- need more info
SEARCH_KB	"Let me look that up" -- search knowledge base
PROVIDE_STEP	"Try this fix" -- give a troubleshooting step
ROUTE_CLAIM	"Transferring to claims" -- insurance/warranty
ESCALATE	"Getting a supervisor" -- beyond normal support
COMPLETE	"Glad I could help" -- problem is resolved

The challenge: The base model (Qwen2.5-1.5B) knows customer support language, but it collapses to one action -- SEARCH_KB ~80% of the time. Even with a carefully engineered prompt, it lacks a consistent decision policy.

The question: Can RL post-training teach the model a better policy than prompt engineering alone?

The Method: GRPO

GRPO = Group Relative Policy Optimization -- a reinforcement learning method from the same family as DeepSeek-R1.

How it works (4 steps, repeated 266 times):

- **Step 1:** Show the model a customer scenario
- **Step 2:** Model generates 4 different responses (K=4 rollouts)
- **Step 3:** Score each response with the reward function (0 to 1)
- **Step 4:** Update model weights: make high-scoring responses more likely, low-scoring less likely

Key components:

- **Base model:** Qwen2.5-1.5B-Instruct (1.5 billion parameters)
- **LoRA adapters:** Only 4M trainable parameters (0.3% of total) -- base model frozen
- **KL regularization:** Prevents the model from drifting too far from its original behavior
- **Training data:** 266 synthetic customer support scenarios across 6 device domains

The Reward Function

The reward function is the "teacher" -- it scores every model response on 7 dimensions. Like a GPA that combines math, English, science into one number.

Component	Weight	What It Grades
Action correctness	25%	Did you pick the right action?
Intent accuracy	20%	Did you diagnose the problem correctly?
Missing info recall	15%	Did you notice what info was missing?
Clarification quality	15%	If you asked a question, was it relevant?
Efficiency	10%	Did you avoid unnecessary questions?
Should-not penalty	10%	Did you avoid harmful actions?
Format compliance	5%	Is the output parseable structured JSON?

Why rule-based, not learned? 392 scenarios is too few to train a robust reward model. Deterministic rules are auditable and interpretable -- you can inspect exactly why any response got its score.

Experimental Design

4 conditions tested under identical evaluation settings (same parser, metrics, reward weights, temperature=0.3):

Condition	Model	Training	Prompt
Random	None	None	None
Zero-shot	Qwen2.5-1.5B	None	Minimal
Engineered	Qwen2.5-1.5B	None	Full engineered
GRPO (RL)	Qwen2.5-1.5B + LoRA	GRPO, 1 epoch	Full engineered

Degenerate Policy Gate (pre-training verification):

Before training, verified that no trivial strategy scores well:

Degenerate Policy	Reward
ALWAYS_ASK	0.500
ALWAYS_SEARCH_KB	0.491
ALWAYS_PROVIDE_STEP	0.455
ALWAYS_ESCALATE	0.435
ALWAYS_COMPLETE	0.365
KEYWORD_STUFFER	0.519

All below 0.55. Any model scoring above 0.616 (engineered baseline) demonstrates genuine learning, not a shortcut.

Results: GRPO Wins on Every Metric

83-scenario held-out test set. Model never saw these during training.

Metric	Random	Zero-shot	Engineered	GRPO (RL)	Delta
Composite Reward	0.441	0.429	0.616	0.657	+0.041
Intent Accuracy	12.0%	0.0%	57.8%	66.3%	+8.5pp
Action Accuracy	21.7%	25.3%	30.1%	34.9%	+4.8pp
Macro F1	0.212	0.150	0.258	0.303	+0.045
Format Compliance	100%	96.4%	98.8%	100%	+1.2pp
Action Entropy	-	1.254	1.012	1.162	+0.150

What each metric means:

- **Composite Reward:** Overall grade (0-1) combining all 7 reward components
- **Intent Accuracy:** Did the model correctly diagnose the customer's problem?
- **Action Accuracy:** Did the model pick the exact right action out of 6?
- **Macro F1:** Average performance across ALL actions (punishes if model ignores rare actions)
- **Format Compliance:** Were all outputs parseable JSON?
- **Action Entropy:** How diverse are the model's choices? (higher = more variety)

Per-Action Performance

Macro F1 is the average of these 6 individual action F1 scores. Precision = "when the model makes this call, how often is it right?" Recall = "of all real cases, how many did the model catch?"

Action	Precision	Recall	F1	Support
ESCALATE	0.80	0.50	0.62	8
SEARCH_KB	0.26	1.00	0.41	17
ROUTE_CLAIM	1.00	0.20	0.33	10
PROVIDE_STEP	0.57	0.21	0.31	19
ASK_CLARIFICATION	0.50	0.09	0.15	23
COMPLETE	0.00	0.00	0.00	6

Key findings:

- **ESCALATE (F1=0.62):** Best performer. When model says "get a supervisor," it's right 80% of the time.
- **ROUTE_CLAIM (Prec=1.00):** Perfect precision -- never sends a non-claim case to claims. But only catches 2 of 10 real claims (low recall).
- **SEARCH_KB (Recall=1.00):** Catches ALL search cases, but floods with false positives (78% of all predictions). This is the SEARCH_KB bias that persists even after GRPO.
- **COMPLETE (F1=0.00):** Total failure -- model never predicts "done." Rarest action, hard to learn from limited data.

What GRPO Improved

Comparing GRPO to the engineered prompt baseline (the best non-RL condition):

Action	Eng. F1	GRPO F1	Change
ROUTE_CLAIM	0.18	0.33	Recall doubled (0.10 -> 0.20)
PROVIDE_STEP	0.24	0.31	Improved across all metrics
ASK_CLARIFICATION	0.08	0.15	Precision improved (0.33 -> 0.50)
SEARCH_KB	0.38	0.41	Recall reached 100%
ESCALATE	0.67	0.62	Slight decrease (-0.05)
COMPLETE	0.00	0.00	No change (both fail)

Per-Domain Reward:

Domain	Engineered	GRPO	Delta
black_screen	0.818	0.832	+0.015
liquid_damage	0.667	0.789	+0.123
lost_stolen	0.735	0.713	-0.022
data_transfer	0.728	0.706	-0.022
charging	0.594	0.659	+0.066
battery_drain	0.551	0.540	-0.011

Largest gain on liquid_damage (+0.123) and charging (+0.066). Minor regression on lost_stolen and data_transfer.

Generalization: Challenge Set

26 scenarios with novel phrasings the model never saw during training. Tests whether the model learned generalizable decision-making, not just pattern matching.

Metric	Test Set (83)	Challenge Set (26)
Composite Reward	0.657	0.576
Intent Accuracy	66.3%	46.2%
Action Accuracy	34.9%	19.2%
Macro F1	0.303	0.107
Format Compliance	100%	100%

Performance drops on novel phrasings (expected). The reward stays above the random baseline (0.441) and format compliance remains perfect. The drop in Macro F1 (0.303 -> 0.107) shows the SEARCH_KB bias intensifies on unfamiliar inputs -- the model retreats to its safe default.

Interpretation: The model learned real decision-making patterns (reward 0.576 > random 0.441), but generalization to completely novel phrasings needs more diverse training data.

Training Details

Parameter	Value
Method	GRPO (Group Relative Policy Optimization)
Base Model	Qwen2.5-1.5B-Instruct
Trainable Parameters	4M (LoRA rank 16, alpha 32)
Training Scenarios	266
Epochs	1
Rollouts per Prompt (K)	4
Learning Rate	1e-5
KL Beta	0.1
Final Training Reward	0.787
Final KL Divergence	0.001
Training Time	17.2 hours
Hardware	Apple M1 Pro (MPS backend)

Why these choices:

- **GRPO over PPO:** No critic network needed. Simpler, cheaper, works with small datasets.
- **Rule-based reward over learned:** 392 scenarios too few for a robust reward model. Rules are auditable.
- **1.5B model:** Feasible on consumer hardware. Forces genuine behavioral learning, not just model scale.
- **1 epoch:** Prevents overfitting on small dataset. Conservative choice -- more epochs could help or hurt.
- **LoRA:** Trains 0.3% of parameters. Base model knowledge preserved, only decision policy changes.

Final Verdict

What GRPO learned:

- Better intent diagnosis -- 66.3% vs 57.8% (+8.5pp)
- Higher precision on minority actions (ROUTE_CLAIM: 100%, ESCALATE: 80%)
- Perfect format compliance -- 100% parseable outputs
- Domain-specific improvement -- liquid_damage +0.123, charging +0.066

What GRPO did NOT fully solve:

- SEARCH_KB collapse -- still predicts SEARCH_KB 78% of the time (ground truth: 20%)
- COMPLETE action -- never predicted (0/6 scenarios)
- Missing information detection -- 0% recall
- Generalization to novel phrasings (challenge set reward drops from 0.657 to 0.576)

Practice beat instructions.

GRPO post-training produced a measurable, consistent improvement over the best prompt engineering baseline. The model learned better problem diagnosis (intent accuracy +8.5pp) and more balanced decision-making (macro F1 +0.045). While the fundamental SEARCH_KB bias persists, this experiment demonstrates that experience-based learning extracts behavioral signal that instruction-based prompting cannot.

Next steps: more training data, more epochs, SEARCH_KB penalty in reward, multi-turn episodes, real customer messages.